

Coin Railz — Demo & Verification Instructions

For Circle Ventures Technical Review February 2026

Quick Start

All endpoints are live at <https://coinrailz.com>. No API key or authentication is required to observe the x402 payment flow. The platform uses the x402 protocol (HTTP 402) — services respond with a payment challenge that includes USDC payment instructions.

1. Verify Discovery Manifests

These endpoints demonstrate that the platform is discoverable by AI agents and ecosystem indexers.

x402 Protocol Manifest

```
curl -s https://coinrailz.com/.well-known/x402.json | jq '.x402'
```

Returns the x402 discovery manifest listing all services with pricing, accepted tokens, and facilitator URL.

A2A Agent Card (Google A2A Protocol)

```
curl -s https://coinrailz.com/.well-known/agent.json | jq '.name, .skills[:3]'
```

Returns the A2A-compliant agent card with service skills.

A2A v0.3 Agent Card

```
curl -s https://coinrailz.com/.well-known/agent-card.json | jq '.name, .description'
```

Machine-Readable Onboarding Guide

```
curl -s https://coinrailz.com/.well-known/agent-instructions.json | jq
'.quickstart'
```

Provides step-by-step instructions for AI agents to begin transacting.

2. Observe the x402 Payment Flow

Step 1: Hit a service without payment

```
curl -s -w "\nHTTP Status: %{http_code}\n" https://coinrailz.com/x402/ping
```

Expected response: HTTP 402 with a JSON payload containing:

- `x402Version: 2`
- `facilitatorUrl` (Coinbase or x402.org facilitator)
- `paymentRequirements` array with USDC amount, receiver address, and supported chains

Step 2: Examine the payment challenge

```
curl -s https://coinrailz.com/x402/ping | jq '{
```

```
  x402Version,
```

```
  facilitatorUrl,
```

```
  requirements: .paymentRequirements[0] | {
```

```
    scheme: .scheme,
```

```
    network: .network,
```

```
    maxAmountRequired: .maxAmountRequired,
```

```
    resource: .resource
```

```
  }
```

```
}'
```

This shows the structured payment challenge that an AI agent would parse to determine how to pay.

Step 3: Try different services

```
# Trading intelligence (low cost)
```

```
curl -s https://coinrailz.com/x402/gas-price-oracle | jq  
'paymentRequirements[0].maxAmountRequired'
```

```
# Prediction markets
```

```
curl -s https://coinrailz.com/x402/polymarket-search | jq  
'paymentRequirements[0].maxAmountRequired'
```

```
# Premium service (high value)
```

```
curl -s https://coinrailz.com/x402/smart-contract-audit | jq  
'paymentRequirements[0].maxAmountRequired'
```

Each returns a 402 challenge with the appropriate USDC price for that service.

3. Verify Multi-Chain Support

The payment challenge includes multiple chain options. To see all supported chains:

```
curl -s https://coinrailz.com/x402/trade-signals | jq  
'paymentRequirements[] | {network, scheme, maxAmountRequired}'
```

You should see payment requirements for multiple networks (Ethereum, Base, Polygon, Arbitrum).

4. Browse the Service Catalog

Full Catalog

```
curl -s https://coinrailz.com/x402/catalog | jq '.services | length'
```

Returns the count of available services.

Bazaar-Compatible Catalog

```
curl -s https://coinrailz.com/api/discovery/resources | jq '{total, services: [.services[:5][] | {id, name, priceUSD}]}'
```

Returns the Coinbase Bazaar-compatible catalog with the first 5 services and pricing.

Payment Documentation (for agents)

```
curl -s https://coinrailz.com/x402/payment-docs | jq '.paymentMethods'
```

Returns documentation of all accepted payment methods.

5. Check Platform Health

```
curl -s https://coinrailz.com/api/monitoring/health | jq '{status, services}'
```

Note: The health endpoint reports operational status. Uptime figures are self-reported telemetry, not independently verified SLA metrics.

Payment Status

```
curl -s https://coinrailz.com/x402/payment-status | jq '{status, recentPayments}'
```

6. Verify Satellite Data Vertical

The satellite data services connect to live NASA and ESA APIs:

```
# Satellite service catalog
```

```
curl -s https://coinrailz.com/api/satellite/catalog | jq '.products[:3]'
```

```
# Fire alerts service (402 challenge)
```

```
curl -s -w "\nHTTP Status: %{http_code}\n"  
https://coinrailz.com/api/satellite/fire-alerts
```

7. Verify Fiat On-Ramp

The Transak-powered fiat on-ramp is accessible at:

```
https://coinrailz.com/buy
```

This page provides a white-label USDC/USDT purchase flow supporting credit card, Apple Pay, and Google Pay across 6 networks.

On-Ramp Quote API

```
curl -s -X POST https://coinrailz.com/api/onramp/quote \  
-H "Content-Type: application/json" \  
-d '{"fiatAmount": 100, "fiatCurrency": "USD", "cryptoCurrency": "USDC",  
"network": "base"}' | jq '.'
```

8. Verify IoT/DePIN Infrastructure

```
# IoT device catalog
```

```
curl -s https://coinrailz.com/api/iot/catalog | jq '.devices[:3]'
```

9. MCP (Model Context Protocol) Discovery

```
curl -s https://coinrailz.com/mcp/services | jq '.services | length'
```

10. Capabilities Endpoint

```
curl -s https://coinrailz.com/api/x402/capabilities | jq '{  
  protocols,  
  paymentMethods,  
  supportedChains,  
  totalServices  
}'
```

Returns a summary of all platform capabilities, supported protocols, and payment methods.

Key Verification Points for Circle

1. **USDC-native:** All payment challenges specify USDC as the settlement token
2. **Multi-chain:** Payment requirements include multiple EVM chains
3. **x402 v2 compliant:** Responses include `x402Version: 2` and Coinbase-compatible facilitator URLs
4. **Production infrastructure:** All endpoints are live, not testnet or mock data
5. **Discovery-ready:** Standard `.well-known` endpoints for ecosystem discoverability
6. **Fiat on-ramp:** Direct USD → USDC purchase flow for end-user acquisition

All endpoints are production. No testnet tokens or simulated responses.